WordPress Performance Incident Runbook

Version 1.0 — Released

Dan Knauss, General Editor

WordPress Performance Incident Runbook

> Status: Released Version: 1.0 Date: 14 June 2026 General Editor: Dan Knauss Currency: Last reviewed against WordPress 7.0 on 2026-06-14.

Use this runbook when a WordPress site has an active or suspected performance incident: elevated TTFB, low cache hit ratio, PHP worker saturation, database slowdown, object-cache failure, WP-Cron or Action Scheduler backlog, external API latency, or a Core Web Vitals regression that is materially affecting users.

This is an execution document. It intentionally differs from the broader guides:

- `wordpress-performance-optimization-checklist.md` explains the general diagnostic flow and transport-layer signal-following discipline.
- `enterprise-performance-operational-checklist.md` expands the same discipline for high-traffic and platform-governed environments.
- `DEVELOPER_REFERENCE.md` explains the underlying APIs, cache layers, and performance model.
- `REFERENCE-WP-Transients-Persistent-Object-Cache.md` explains transients, persistent object cache behavior, and cache-stampede risk.

Table of Contents

Environment placeholders

Customize these before running commands:

```
export WP_PATH="[CUSTOMIZE: /path/to/wordpress]"
export SITE_URL="[CUSTOMIZE: https://example.com]"
export TARGET_URL="[CUSTOMIZE: https://example.com/problem-url/]"
export GOOD_URL="[CUSTOMIZE: https://example.com/known-good-url/]"
export INCIDENT_ID="[CUSTOMIZE: YYYYMMDD-short-name]"
export EVIDENCE_DIR="./incident-${INCIDENT_ID}"
mkdir -p "$EVIDENCE_DIR"

# SETUP / LOCAL WRITE: creates only the local evidence directory.
# READ-ONLY: remaining setup commands inspect the selected WordPress site.
```

```
CURL_TIMEOUT_ARGS=(--connect-timeout 5 --max-time 30)

# Site-specific WP-CLI arguments. Keep --url for multisite and harmless single-site parity.
WP_CLI_BASE_ARGS=(--path="$WP_PATH")
WP_CLI_SITE_ARGS=(--path="$WP_PATH" --url="$SITE_URL")

# Derive the affected site's options table instead of hand-typing wp_options/wp_2_options.
OPTIONS_TABLE="$(wp "${WP_CLI_SITE_ARGS[@]}" db prefix)options"
wp "${WP_CLI_SITE_ARGS[@]}" db query "SHOW TABLES LIKE '${OPTIONS_TABLE}';"

# Authenticated/dynamic request placeholders. Leave empty for anonymous checks.
COOKIE_JAR="[CUSTOMIZE: /path/to/auth-cookies.txt or empty]"
AUTH_HEADER="[CUSTOMIZE: Authorization: Bearer token or empty]"
EXPECTED_STATUS="[CUSTOMIZE: expected HTTP status, e.g. 200]"
CURL_AUTH_ARGS=()
# CURL_AUTH_ARGS=(-b "$COOKIE_JAR")
# CURL_AUTH_ARGS=(-H "$AUTH_HEADER")
```

Assumptions:

- WP-CLI is available as `wp`.
- Commands run with a user that can read the WordPress install and, where needed, query the database.
- Production write operations require explicit incident lead approval.
- Site-specific WP-CLI commands use `WP_CLI_SITE_ARGS`; for multisite this targets the affected blog with `--url="$SITE_URL"`. Profiling commands that intentionally request `TARGET_URL` use `WP_CLI_BASE_ARGS` plus the command-specific `--url="$TARGET_URL"` to avoid passing two `--url` values.
- `OPTIONS_TABLE` is derived from `wp db prefix` for the selected site and verified before raw SQL is used. For network-wide checks, label the command as network-wide before running it.

Severity triggers

Severity	Trigger examples	Response expectation
SEV1	Public site broadly unavailable, checkout/account impossible, sustained origin overload, database unavailable	Immediate incident response, platform/hosting escalation, change freeze
SEV2	Major template or admin workflow slow for many users, cache hit ratio collapse, PHP workers saturated	Incident response with focused mitigation and stakeholder updates
SEV3	Degraded subset of URLs, single integration slow, Core Web Vitals regression without outage	Triage, measured fix, normal change controls unless risk grows

Communications cadence

- Communications owner: [CUSTOMIZE: role/name assigned in the incident channel].
- SEV1: update stakeholders every 15 minutes until mitigated, then every 30 minutes until resolved.
- SEV2: update stakeholders every 30 minutes until mitigated, then hourly until resolved.
- SEV3: update at start, material state changes, mitigation, and closeout.
- Minimum update contents: severity, user impact, affected paths, current hypothesis, mitigation in progress, next decision time, and risks or help needed.
- Channels: [CUSTOMIZE: incident channel, status page, customer-support channel, executive/internal update path].

Safety rules

- Start read-only. Capture headers, timings, logs, and queue state before changing anything.
- Do not globally flush CDN/page/object caches unless the incident lead approves and the blast radius is understood.

- Do not disable plugins, change PHP versions, alter database indexes, or purge queues in production without rollback and approval.
- Do not cache authenticated, personalized, cart/checkout, preview, or admin responses as public HTML.
- Prefer bypassing, isolating, or rate-limiting one bad path over broad site-wide changes.
- Preserve evidence: save command output, timestamps, and before/after measurements.

First 10 minutes

1. Assign roles: incident lead, WordPress engineer, platform/hosting engineer, communications owner, and stakeholder update cadence.
2. Confirm user impact and severity.
3. Freeze unrelated deploys, cache-rule changes, and plugin/theme updates.
4. Capture response headers and timings for one affected URL and one known-good URL into `EVIDENCE_DIR`.
5. Determine whether the affected request is served from edge/page cache or reaches origin/PHP.
6. Pick the matching procedure below and proceed.

Procedure 1: Initial performance incident triage

Procedure Metadata

- Owner: [CUSTOMIZE: Incident lead]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 10 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Establish the incident scope, preserve evidence, and decide which deeper procedure to run next.

Prerequisites

- Shell access with `curl`.
- WP-CLI access for the target WordPress install.
- Known affected URL, known-good URL, and expected user state: anonymous, logged-in, admin, REST, AJAX, cron, or checkout/account.

Commands

```
date -u
printf 'Incident: %s\nSite: %s\nTarget: %s\n' "$INCIDENT_ID" "$SITE_URL" "$TARGET_U

curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-target.txt" \
  -w 'target dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect}
   "$TARGET_URL"

curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-known-good.txt" \
  -w 'known_good dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconn
   "$GOOD_URL"

sed -n '1,80p' "$EVIDENCE_DIR/headers-${INCIDENT_ID}-target.txt"
sed -n '1,80p' "$EVIDENCE_DIR/headers-${INCIDENT_ID}-known-good.txt"

wp "${WP_CLI_SITE_ARGS[@]}" core version
wp "${WP_CLI_SITE_ARGS[@]}" option get siteurl
wp "${WP_CLI_SITE_ARGS[@]}" option get home
```

Expected Output

- `curl` reports HTTP status, TTFB, and total time for both affected and known-good URLs.
- Response headers show whether the response was a cache `HIT`, `MISS`, `BYPASS`, `DYNAMIC`, or origin response according to the site/CDN conventions.
- WP-CLI returns the WordPress version and expected site URLs.

Rollback

No rollback is required. This procedure is read-only.

Verification

```
test -s "$EVIDENCE_DIR/headers-${INCIDENT_ID}-target.txt" && test -
s "$EVIDENCE_DIR/headers-${INCIDENT_ID}-known-good.txt" && echo "headers captured"
wp "${WP_CLI_SITE_ARGS[@]}" core is-installed && echo "wp-cli can inspect the instal
```

Expected: both checks print success messages.

Escalate If

- WP-CLI cannot access the install during a production incident.
- The site returns 5xx, connection failures, or TLS errors for broad public traffic.
- Headers indicate private/authenticated content may be cached publicly.

Procedure 2: High TTFB on anonymous public pages

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 20 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Determine whether high anonymous-page TTFB is caused by cache miss/bypass, transport/edge delay, origin/PHP execution, database work, object-cache misses, or external dependencies.

Prerequisites

- Affected URL is safe to request anonymously.
- CDN/page-cache header conventions are known.
- Permission to inspect cache rules and origin metrics.

Commands

```
for i in 1 2 3; do
  curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-anon-${i}.txt" \
   -w "run:${i} dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconne
    "$TARGET_URL"
done

grep  -E  '^(HTTP/|cache-control:|Cache-Control:|age:|Age:|x-cache|X-
Cache|cf-cache-status|CF-Cache-Status|x-redirect-by|X-Redirect-
By|set-cookie:|Set-Cookie:)'  "$EVIDENCE_DIR"/headers-${INCIDENT_ID}-
anon-*.txt || true

wp "${WP_CLI_SITE_ARGS[@]}" cron event list --due-now

# WRITE — approval required. Optional cleanup; run only after incident lead approval
# This can add database load and remove evidence of transient churn.
# wp "${WP_CLI_SITE_ARGS[@]}" transient delete --expired
```

Expected Output

- Repeated anonymous requests should show whether TTFB improves after warming.
- Cache headers should explain `HIT`, `MISS`, `BYPASS`, `DYNAMIC`, or equivalent behavior.
- `Set-Cookie` on anonymous cacheable pages is a red flag because it can fragment or bypass HTML cache.

Rollback

No rollback is required for the default read-only checks. If the optional expired-transient cleanup is approved and run, there is no practical rollback for deleted expired transient rows; rely on application regeneration and continue monitoring. If an approved cache-rule change is made outside this procedure, restore the previous cache rule from the CDN/host change history.

Verification

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-verify.txt" \
```

```
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
  "$TARGET_URL"

grep     -E     '^(HTTP/|age:|Age:|x-cache|X-Cache|cf-cache-status|CF-
Cache-Status|cache-control:|Cache-Control:)'   "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-verify.txt" || true
```

Expected: cacheable anonymous pages show an intentional cache status and improved TTFB after warming.

Escalate If

- Cacheable anonymous pages consistently bypass cache without an intentional reason.
- Cache headers vary by marketing query strings, unnecessary cookies, or inconsistent device/geo keys.
- Origin TTFB remains high after confirming cache hit behavior.
- A cache purge or rule change is needed during peak traffic.

Procedure 3: High TTFB on logged-in, admin, REST, or AJAX requests

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 30 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Diagnose slow dynamic requests that cannot be solved by public full-page caching.

Prerequisites

- A target URL, REST route, AJAX action, or admin screen has been identified.
- Authentication method is available for deeper tooling if needed, and `CURL_AUTH_ARGS` is set when verifying logged-in/admin/REST/AJAX behavior.

- WP-CLI profile and doctor packages may not be installed; check availability before relying on them.

Commands

```
if wp "${WP_CLI_SITE_ARGS[@]}" cli has-command profile; then
  echo "profile available"
 # Use WP_CLI_BASE_ARGS here because wp profile uses --url as the profiled request U
 # TARGET_URL must be the affected site URL, which also selects the correct multisit
  wp "${WP_CLI_BASE_ARGS[@]}" profile stage --url="$TARGET_URL"
  wp "${WP_CLI_BASE_ARGS[@]}" profile hook --url="$TARGET_URL" --all
else
  echo "profile not installed"
fi

if wp "${WP_CLI_SITE_ARGS[@]}" cli has-command doctor; then
  echo "doctor available"
  wp "${WP_CLI_SITE_ARGS[@]}" doctor check
else
  echo "doctor not installed"
fi

# Always safe baseline checks:
wp "${WP_CLI_SITE_ARGS[@]}" plugin list --status=active
wp "${WP_CLI_SITE_ARGS[@]}" theme list --status=active
```

Expected Output

- `wp profile stage` identifies whether bootstrap, main query, template, or shutdown dominates.
- `wp profile hook` identifies slow hooks/callbacks.
- `wp doctor check` flags common foot-guns where available.
- Active plugin/theme inventory supports rollback and ownership decisions.

Rollback

No rollback is required for profiling. If an approved mitigation disables a plugin/module or changes configuration, restore the previous plugin/configuration state from the incident log or deployment system.

Verification

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" "${CURL_AUTH_ARGS[@]}" -o "$EVIDENCE_DIR/body-
${INCIDENT_ID}-dynamic-verify.txt" -D "$EVIDENCE_DIR/headers-${INCIDENT_ID}-
dynamic-verify.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"

if ! grep -E "^HTTP/.* ${EXPECTED_STATUS}" "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-dynamic-verify.txt"; then
  echo "Unexpected status for authenticated/dynamic verification" >&2
  exit 1
fi
# Optional body marker check for authenticated/admin/REST/AJAX responses:
# grep -F "[CUSTOMIZE: expected body marker]" "$EVIDENCE_DIR/body-
${INCIDENT_ID}-dynamic-verify.txt"
```

Expected: TTFB and total time improve for the same URL, user state, authentication method, and expected HTTP status/body marker, or the dominant bottleneck is identified for escalation.

Escalate If

- Profiling points to a third-party plugin, theme, or external API without a safe local mitigation.
- PHP workers are saturated or requests queue at the host level.
- Admin/editor workflows are blocked for publishing, ecommerce, membership, or support teams.
- WordPress 7.0 admin/editor compatibility issues are suspected after an upgrade.

Procedure 4: Database slowdown, autoload bloat, or expensive queries

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress/database engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 20 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Identify whether database time is dominated by slow queries, too many queries, inefficient query shape, or autoloaded option bloat.

Prerequisites

- Database read access through WP-CLI.
- Approval before changing options, deleting data, or adding indexes.
- Current backup/snapshot before any write operation.

Commands

```
wp "${WP_CLI_SITE_ARGS[@]}" db query "SELECT option_name, LENGTH(option_value) AS s
on') ORDER BY size DESC LIMIT 20;"

wp "${WP_CLI_SITE_ARGS[@]}" db query "SELECT SUM(LENGTH(option_value)) AS autoload_
on');"

wp "${WP_CLI_SITE_ARGS[@]}" db size --tables
```

Expected Output

- Autoload queries include both legacy `yes` and modern WordPress 6.6+ values: `on`, `auto`, and `auto-on`.
- Large autoloaded options are identified by name and size.
- Large tables are identified for deeper query review.

Rollback

If an approved autoload change is made, record the previous autoload value first and restore it if behavior regresses.

```
# Read before changing.
wp "${WP_CLI_SITE_ARGS[@]}" db query "SELECT option_name, autoload FROM ${OPTIONS_T

# ROLLBACK — approval required. Choose the previous value captured above, then uncom
# wp "${WP_CLI_SITE_ARGS[@]}" option set-autoload "[CUSTOMIZE: option_name]" on
```

Verification

```
wp "${WP_CLI_SITE_ARGS[@]}" db query "SELECT SUM(LENGTH(option_value)) AS autoload_
on');"

curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-db-verify.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"
```

Expected: autoload footprint decreases if that was the target, and the affected request improves without functional regression.

Escalate If

- Database CPU, I/O, or lock waits are elevated at the host/provider level.
- Slow queries require schema/index changes in production.
- A large option is owned by a plugin/theme and changing autoload state could affect correctness.
- Queries use broad post meta, taxonomy exclusions, `LIKE`, or unbounded result sets at production scale.

Procedure 5: Object cache failure or cache stampede

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress/platform engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 20 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Determine whether a persistent object cache failure, poor hit rate, key misuse, or cache stampede is causing origin/database overload.

Prerequisites

- Access to WordPress filesystem and cache service metrics.
- Knowledge of whether Redis, Memcached, Object Cache Pro, or another drop-in is used.
- Approval before restarting cache services or flushing object cache.

Commands

```
test -f "$WP_PATH/wp-content/object-cache.php" && echo "object-cache.php drop-in present" || echo "no persistent object-cache drop-in found"

wp "${WP_CLI_SITE_ARGS[@]}" cache type || true
wp "${WP_CLI_SITE_ARGS[@]}" cache get non_existing_key incident_probe || true

# WRITE — approval required. Optional cleanup; run only after incident lead approval
# This can add database load and remove evidence of transient churn.
# wp "${WP_CLI_SITE_ARGS[@]}" transient delete --expired
```

Expected Output

- `object-cache.php` presence confirms a persistent object-cache drop-in is installed, but service metrics must confirm health.
- Cache command behavior identifies whether WP-CLI can interact with the configured object cache.
- Expired transient cleanup is intentionally not part of the default read-only evidence gathering. If approved, it is only a cleanup action and should not be treated as a primary incident fix.

Rollback

Do not flush object cache as rollback. If optional expired-transient cleanup is approved and run, there is no practical rollback for deleted expired transient rows; rely on application regeneration and continue monitoring. If an approved cache-service restart or configuration change worsens the incident, restore the previous service/configuration state through the hosting/platform control plane.

```
# PLUGIN-DEPENDENT / ROLLBACK — approval required. Uncomment only for the cache plug
# wp "${WP_CLI_SITE_ARGS[@]}" redis status
```

```
# wp "${WP_CLI_SITE_ARGS[@]}" redis enable
# wp "${WP_CLI_SITE_ARGS[@]}" redis disable
```

Verification

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-cache-verify.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"
```

Expected: database load, cache errors, or TTFB improve after the approved mitigation.

Escalate If

- Cache service is unreachable, evicting heavily, or out of memory.
- `alloptions` or transient stampedes are suspected under concurrent traffic.
- Code relies on `wp_cache_add()` locks without a shared persistent object cache with atomic add semantics.
- A global object-cache flush is proposed during high traffic.

Procedure 6: WP-Cron or Action Scheduler backlog

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress operations engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 20 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Identify whether due cron events, failed scheduled actions, or overlapping background jobs are causing request-time latency or delayed business workflows.

Prerequisites

- WP-CLI access.
- Approval before disabling WP-Cron, changing schedules, or canceling/deleting jobs.
- Knowledge of whether WooCommerce or Action Scheduler is active.

Commands

```
wp "${WP_CLI_SITE_ARGS[@]}" cron event list --due-now
wp "${WP_CLI_SITE_ARGS[@]}" cron event list --fields=hook,next_run_relative,recurr

# PLUGIN-DEPENDENT / READ-ONLY - uncomment if Action Scheduler WP-
CLI commands are available:
# wp "${WP_CLI_SITE_ARGS[@]}" action-scheduler list --status=pending -
-per-page=20
# wp "${WP_CLI_SITE_ARGS[@]}" action-scheduler list --status=failed -
-per-page=20
```

Expected Output

- Due cron events should clear when the runner works.
- Repeated due events, long-running hooks, or growing queues indicate a backlog.
- Failed Action Scheduler jobs identify plugin/business workflow ownership.

Rollback

If `DISABLE_WP_CRON` was added and the external runner is not working, remove or comment out the constant and clear OPcache if required by the host.

```
// Rollback in wp-config.php if the external runner fails:
// define( 'DISABLE_WP_CRON', true );
```

Verification

```
wp "${WP_CLI_SITE_ARGS[@]}" cron event list --due-now

# WRITE — approval required. Running due events can send email, process queues,
# call external services, and add PHP/database load. Prefer a specific known-
safe
# hook over all due events when possible.
```

```
# wp "${WP_CLI_SITE_ARGS[@]}" cron event run "[CUSTOMIZE: hook_name]" --due-now
# wp "${WP_CLI_SITE_ARGS[@]}" cron event run --due-now --quiet
# wp "${WP_CLI_SITE_ARGS[@]}" cron event list --due-now
```

Expected: read-only verification shows whether due events are decreasing under the normal runner. If approved cron execution is run, due events decrease or clear and business workflows such as scheduled posts, cleanup jobs, and ecommerce queues resume.

Escalate If

- The queue grows after the runner is confirmed.
- Jobs fail repeatedly for the same hook or plugin owner.
- Running due events causes PHP/database saturation.
- A high-traffic site still relies only on request-triggered WP-Cron.

Procedure 7: External API, AI connector, or third-party latency

Procedure Metadata

- Owner: [CUSTOMIZE: WordPress engineer / integration owner]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 15 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Identify whether external HTTP calls, WordPress 7.0 AI/connectors integrations, payment/search/CRM APIs, analytics, or embed providers are slowing critical requests.

Prerequisites

- Affected request path is known.
- Integration owners and API status pages are known.
- A safe fallback or feature flag is identified before disabling integrations.

Commands

```
wp "${WP_CLI_SITE_ARGS[@]}" plugin list --status=active --fields=name,status,versi

curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-external.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"
```

Expected Output

- Active plugin inventory identifies likely integration owners.
- Timings show whether the request is slow enough to justify deeper APM/Query Monitor inspection.
- APM traces, if available, should separate HTTP API time from PHP, database, and cache time.

Rollback

If an approved feature flag, connector setting, or plugin configuration change is made, restore the prior setting from the incident log or configuration management system.

```
# PLUGIN-DEPENDENT / ROLLBACK — approval required. Uncomment only for the specific p
# wp "${WP_CLI_SITE_ARGS[@]}" option get "[CUSTOMIZE: option_name]"
# wp "${WP_CLI_SITE_ARGS[@]}" option update "[CUSTOMIZE: option_name]" "[CUSTOMIZE
```

Verification

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-external-verify.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"
```

Expected: request time improves or APM confirms the external dependency is no longer in the critical path.

Escalate If

- The dependency is required for checkout, login, publishing, search, or compliance.
- Timeouts/retries amplify traffic or worker saturation.

- AI/connector workflows expose credential, privacy, or rate-limit risk.
- No safe degraded mode exists.

Procedure 8: Core Web Vitals regression

Procedure Metadata

- Owner: [CUSTOMIZE: Frontend/WordPress engineer]
- Last Tested: [CUSTOMIZE: YYYY-MM-DD]
- Review Cadence: Quarterly
- Estimated Time: 15 minutes
- Last Drill Date: [CUSTOMIZE: YYYY-MM-DD / N/A]

Purpose

Triage a user-facing LCP, INP, or CLS regression without confusing frontend symptoms with backend, cache, or transport causes.

Prerequisites

- Affected template or URL set is known.
- Field data source is identified: CrUX, RUM, Search Console, analytics, or APM browser agent.
- Browser tooling may be required for final diagnosis; if unavailable, capture back-end/transport evidence first and escalate for browser inspection with requested artifacts: DevTools trace, Lighthouse or WebPageTest report, RUM segment, device/viewport class, and before/after screenshots when visual instability is involved.

Commands

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-cwv.txt" \
  -w 'dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%
   "$TARGET_URL"

grep  -E  '^(HTTP/|cache-control:|Cache-Control:|age:|Age:|x-cache|X-
Cache|cf-cache-status|CF-Cache-Status|content-type:|Content-Type:)' "$EVIDENCE_DI
${INCIDENT_ID}-cwv.txt" || true
```

Expected Output

- Transport and TTFB data clarify whether poor LCP may start with slow document delivery.
- Cache headers clarify whether public HTML and assets are delivered efficiently.
- Further LCP/INP/CLS diagnosis usually requires browser traces, RUM, lab tooling, device/viewport segmentation, and screenshots for visual instability.

Rollback

If the regression follows a deploy, rollback through the normal release system after incident lead approval. If it follows a CDN/cache/optimization plugin change, restore the previous configuration.

Verification

```
curl -sS "${CURL_TIMEOUT_ARGS[@]}" -o /dev/null -D "$EVIDENCE_DIR/headers-
${INCIDENT_ID}-cwv-verify.txt" \
  -w 'ttfb:%{time_starttransfer} total:%{time_total} code:%{http_code}\n' \
   "$TARGET_URL"
```

Expected: backend/transport signals are healthy before handing off to browser-level investigation; after frontend mitigation, field or lab metrics improve for the same template/device class.

Escalate If

- LCP remains poor despite fast cached TTFB.
- INP regression correlates with third-party scripts, page-builder runtime, or new editor/frontend assets.
- CLS is caused by ads, embeds, consent banners, notices, or late-injected personalized content.
- Browser automation, DevTools traces, Lighthouse/WebPageTest output, RUM segmentation, screenshots, or visual inspection are required and not available in the current session.

Recovery verification checklist

- ☐ Same URL, same user state, and same cache state were tested before and after mitigation.
- ☐ Public cache behavior is intentional and safe.
- ☐ Authenticated, personalized, checkout, account, admin, preview, and REST/AJAX paths still behave correctly.
- ☐ PHP errors, database errors, and cache-service errors are not increasing.
- ☐ Cron and queues are clearing if they were part of the incident.
- ☐ APM/RUM/headers/logs show recovery, not just one successful manual request.
- ☐ Stakeholders know what changed, what remains under watch, and when the next update or closeout will happen.

Post-incident follow-up

- ☐ Write a short timeline with symptoms, trigger, detection, mitigation, stakeholder communications, and verification.
- ☐ Open follow-up issues for root-cause fixes that were not safe during the incident.
- ☐ Add or tune alerts for the earliest reliable signal.
- ☐ Review cache rules, object-cache health, slow queries, cron schedules, and external dependencies touched during the incident.
- ☐ Update this runbook if responders had to improvise.